

FastAPI Banking API: 25 MCQ

Medium to High difficulty · each question is followed by its correct answer and an explanation

Format. Every question is laid out as: **question and four choices**, then on a new line **the correct answer**, then **explanation** paragraphs: why the answer is right and why each other choice is wrong. All answers were verified by running the project (FastAPI 0.141, Pydantic 2.13, SQLAlchemy 2.0). Many questions describe a *small change* to the code, so learners have to reason about what the code does, not recall a slide.

Q1. Account 999 does not exist. A client sends `POST /accounts/999/deposit` with body `{"amount": -5}`. What is the response?

MEDIUM
Service rules

- A. 422, because the `float` field rejects negative numbers
- B. 400 with detail "Deposit amount must be positive"
- C. 404, because path parameters are resolved before the body is used
- D. 404 with detail "Account not found"

Correct answer: B. 400 with detail "Deposit amount must be positive"

Explanation. `deposit()` checks `amount <= 0` **before** calling `repo.get_by_id`, so the amount error wins. `amount: float` has no `gt=0` constraint, so Pydantic accepts -5. Verified.

Why the other options are wrong.

- A. `float` accepts negative numbers; only `Field(gt=0)` would make Pydantic reject -5.
- C. Path conversion only turns "999" into an int. Whether the account exists is decided later, inside the service, after the amount check.
- D. The lookup never happens: the amount check fails first and raises `ValueError`, which the router maps to 400.

Q2. During a single request to `POST /accounts/1/withdraw`, how many times is `get_db()` executed?

MEDIUM
Dependency Injection

- A. Twice; once for `AccountService` and once for `AccountRepository`
- B. Once at application startup; that session is then shared by all requests
- C. Never; `AccountRepository` opens its own session through `SessionLocal()`
- D. Once; FastAPI caches a dependency's value for the duration of one request

Correct answer: D. Once; FastAPI caches a dependency's value for the duration of one request

Explanation. Only the repository depends on `get_db`, and dependency results are cached per request (`use_cache=True`). A new request gets a new session. Measured with a counting override: exactly 1 call per request.

Why the other options are wrong.

- A. `AccountService` does not depend on `get_db` directly, and even if two dependencies needed it, the cached value would be reused.
- B. A session per application would be shared across threads and requests; `get_db` is a per-request generator, not a startup hook.
- C. The repository receives its session through `Depends(get_db)`; it never calls `SessionLocal()` itself.

Q3. In the router, `service.withdraw(...)` returns `None`. What reaches the client?

MEDIUM

Error handling

```
try:
    updated = service.withdraw(account_id, request.amount)
    if not updated:
        raise HTTPException(status_code=404, detail="Account not found")
    return updated
except ValueError as e:
    raise HTTPException(status_code=400, detail=str(e))
```

- A. 500, because an exception raised inside `try` is treated as unhandled
- B. 404 with detail "Account not found"
- C. 200 with a `null` body
- D. 400 with detail "Account not found", because the `except` block converts it

Correct answer: B. 404 with detail "Account not found"

Explanation. `HTTPException` is not a subclass of `ValueError`, so it passes through the `except ValueError` clause and FastAPI returns 404.

Why the other options are wrong.

- A. Raising inside `try` is normal Python. The exception simply propagates, and FastAPI knows how to turn `HTTPException` into a response.
- C. `return updated` is skipped because the `raise` happens first, but a raised `HTTPException` produces its own error response, not a 200.
- D. The `except` block only catches `ValueError`; `HTTPException` is a different class and is not intercepted.

Q4. A developer changes `except ValueError as e:` to `except Exception as e:` in the snippet from the previous question. Now a client withdraws 10 from non-existent account 999. What is returned?

HIGH

Error handling

- A. 500, because re-raising inside an `except` block is not allowed
- B. 422: response doesn't match `AccountOut`
- C. 400 with detail "404: Account not found"
- D. 404 with detail "Account not found", exactly as before

Correct answer: C. 400 with detail "404: Account not found"

Explanation. `except Exception` now also catches the `HTTPException(404)` and re-raises it as a 400 whose detail is `str(e)`; for Starlette's `HTTPException` that string is "404: Account not found". Verified. Lesson: catch the narrowest exception type.

Why the other options are wrong.

- A. Raising a new exception inside `except` is valid Python (it is exactly what the original 400 branch does).
- B. No `AccountOut` validation happens because no account object is returned; the request ends with an HTTP error.
- D. It is no longer the same as before: `except Exception` now catches the `HTTPException` as well.

Q5. What does `curl -i http://127.0.0.1:8000/accounts` (no trailing slash) return? **MEDIUM**

Routing

- A. 404 Not Found, because only `/accounts/` is registered
- B. 405 Method Not Allowed
- C. 200 with the account list; FastAPI treats both spellings as one route
- D. 307 Temporary Redirect to `/accounts/`

Correct answer: D. 307 Temporary Redirect to `/accounts/`

Explanation. Routes are registered with a trailing slash ("`/`" under prefix `/accounts`). Starlette's `redirect_slashes` answers the other spelling with 307. Verified.

Why the other options are wrong.

- A. The route exists as `/accounts/`, and Starlette redirects the slash-less path instead of returning 404.
- B. 405 is returned when a path exists but not for that method. Here the redirect happens first.
- C. The two paths are different routes; FastAPI does not merge them, it redirects.

Q6. A client calls `POST /accounts/1/withdraw?amount=50` with no request body. What happens? **MEDIUM**

Request parsing

- A. 422: the endpoint needs `AmountRequest` in the JSON body
- B. 400 "Withdraw amount must be positive", because `amount` defaults to 0
- C. 405: POST can't take query parameters
- D. 200; FastAPI falls back to the query string when the body is empty

Correct answer: A. 422: the endpoint needs `AmountRequest` in the JSON body

Explanation. A parameter typed as a Pydantic model is read from the body. The error is `type: missing, loc: ["body"]`. Verified.

Why the other options are wrong.

- B. `AmountRequest.amount` has no default, so the missing body is a validation error (422), not a zero amount.
- C. HTTP allows query strings on any method; the request fails because the required body is missing.
- D. FastAPI never switches sources: a Pydantic-model parameter is always read from the body.

Q7. A client sends `POST /accounts/` with `{"owner_name": "Ravi", "balance": 5000}`. What is the result? **MEDIUM**

Schemas

- A. 422, because `AccountCreate` does not declare a `balance` field
- B. 200; the account is created with balance `5000.0` since the model has that column
- C. 400, because the service refuses clients that try to set a balance
- D. 200; the account is created with balance `0.0` and the extra field is ignored

Correct answer: D. 200; the account is created with balance `0.0` and the extra field is ignored

Explanation. Pydantic's default is `extra="ignore"`, and the service builds `Account(owner_name=..., balance=0.0)` explicitly. Verified.

Why the other options are wrong.

- A. Pydantic ignores unknown fields by default (`extra="ignore"`); it would only reject them with `extra="forbid"`.
- B. The client's `balance` never reaches the model: `AccountCreate` drops it, and the service sets `balance=0.0` explicitly.
- C. The service has no such check. It never sees the extra field because the schema already dropped it.

Q8. The class default is `APP_NAME = "Personal Banking API"`; `.env` contains `APP_NAME="Personal Banking API (Local Server)"`. The server is started from `banking_api/` after `export APP_NAME=""`. What is `settings.APP_NAME`?

HIGH
Configuration

- A. `"Personal Banking API (Local Server)"`
- B. `"Personal Banking API"`, because an empty variable disables `.env` and uses the default
- C. `""` (an empty string)
- D. Startup fails with a `ValidationError`, because an empty string is not accepted

Correct answer: A. `"Personal Banking API (Local Server)"`

Explanation. `env_ignore_empty=True` makes the empty environment variable count as *not set*, so the next source, `.env`, supplies the value. Verified.

Why the other options are wrong.

- B. An empty variable does not disable `.env`; it is simply ignored and `.env` still supplies the value.
- C. With `env_ignore_empty=True`, an empty environment variable counts as not set, so it cannot win.
- D. Nothing is validated from the empty value, because it is ignored before validation.

Q9. You add `app/models/transaction.py` with `class Transaction(Base)` and `__tablename__ = "transactions"`, but no file imports it. You restart the app. What happens?

HIGH
SQLAlchemy metadata

- A. Startup fails with an `ImportError`, because the module is never used
- B. No `transactions` table; `create_all` only sees models that were imported
- C. The table is created lazily the first time a `Transaction` is queried
- D. The table is created; every subclass of `Base` inside the package is discovered automatically

Correct answer: B. No `transactions` table; `create_all` only sees models that were imported

Explanation. A table is registered on `Base.metadata` when the class body executes, i.e. on import. Verified: `create_all` before importing the model creates zero tables.

Why the other options are wrong.

- A. An unimported module is never executed, so it cannot raise an `ImportError`.
- C. SQLAlchemy never creates tables on query. The query would fail because the table does not exist.
- D. SQLAlchemy has no package scanning; a model exists in `Base.metadata` only after its class is executed (imported).

Q10. You add `email = Column(String)` to `Account` and restart with the existing `bank.db`. You then call `GET /accounts/1`. What happens?

MEDIUM
Schema changes

- A. The request returns 200, because SQLAlchemy skips columns missing in the database
- B. `create_all` adds the `email` column with NULL values and the request returns 200
- C. The table isn't altered; the SELECT names a missing `email` column and fails
- D. `create_all` drops and recreates the table, so the request returns 404

Correct answer: C. The table isn't altered; the SELECT names a missing `email` column and fails

Explanation. `create_all` only issues `CREATE TABLE` for missing tables; it never alters existing ones. The SELECT now names `accounts.email` and SQLite raises *no such column* (500). Use Alembic migrations.

Why the other options are wrong.

- A. SQLAlchemy builds the SELECT from the model, so it names the missing column, and SQLite rejects it.
- B. `create_all` never alters an existing table; it only creates missing ones.
- D. `create_all` never drops tables. Existing data stays in `bank.db`.

Q11. Why does `database.py` pass `{"check_same_thread": False}` for SQLite URLs?

MEDIUM
Database engine

- A. It is required by `async def` endpoints, which run across multiple event loops
- B. It lets several uvicorn worker processes write to the same SQLite file at the same time
- C. It switches SQLite into autocommit mode so `commit()` calls become optional
- D. Sync `def` endpoints run in a thread pool, so a connection can cross threads

Correct answer: D. Sync `def` endpoints run in a thread pool, so a connection can cross threads

Explanation. Python's `sqlite3` driver refuses cross-thread use by default. FastAPI executes sync endpoints in a thread pool; each request still gets its own session.

Why the other options are wrong.

- A. These endpoints are plain `def`. And `async` code runs on a single event loop, not several.
- B. SQLite's thread check has nothing to do with multiple processes; SQLite still allows only one writer at a time.
- C. Autocommit is unrelated. `commit()` is still required, and the repository still calls it.

Q12. `AccountRepository.update()` calls `self.db.commit()` without ever calling `self.db.add(account)`. Why is the new balance saved?

HIGH
Sessions

- A. The session loaded it, so it tracks the change and `commit()` flushes it
- B. Assigning to `account.balance` executes an UPDATE immediately; `commit()` only ends the transaction
- C. `refresh()` writes the object back before reloading it
- D. It is not saved; the returned balance is only the in-memory value and is lost after the request

Correct answer: A. The session loaded it, so it tracks the change and `commit()` flushes it

Explanation. Objects loaded via a session are *persistent* in it. Changing an attribute marks them dirty; with `autoflush=False` nothing is sent until `commit()`, which flushes and commits.

Why the other options are wrong.

- B. With `autoflush=False`, assigning an attribute sends no SQL. The UPDATE is sent during `commit()`.
- C. `refresh()` does the opposite: it reads from the database and overwrites the in-memory values.
- D. The change is saved: `commit()` writes the UPDATE, which is why the next GET shows the new balance.

Q13. A client sends `POST /accounts/1/deposit` with `{"amount": "abc"}`. Which statement describes what happens inside the server?

HIGH
Request lifecycle

- A. The endpoint runs and `service.deposit` raises `ValueError`, producing a 400
- B. Validation fails before any dependency is resolved, so `get_db()` is never even called
- C. Dependencies run, the endpoint body is skipped, 422 is sent, the session is closed
- D. 422 is returned, but the session stays open because the endpoint never ran

Correct answer: C. Dependencies run, the endpoint body is skipped, 422 is sent, the session is closed

Explanation. FastAPI resolves dependencies and validates parameters in the same phase; only the endpoint function is skipped on errors, and `finally: db.close()` still runs. Verified with spies: `get_db → service_init → close`, then 422.

Why the other options are wrong.

- A. The endpoint body never runs, because body validation fails and FastAPI returns 422 instead.
- B. FastAPI resolves dependencies before it validates the body, so `get_db()` does run. Verified with a spy.
- D. Dependency cleanup always runs. The `finally: db.close()` in `get_db` still executes.

Q14. On a new account (balance 0.0) you deposit 0.1 and then 0.2. What balance does the second response show?

MEDIUM
Data types

- A. `0.30000000000000004`
- B. `0.3`
- C. 422, because floats with long fractions are rejected
- D. `0.30`, rounded by `response_model` to two decimals

Correct answer: A. `0.30000000000000004`

Explanation. Binary floating point cannot represent 0.1 or 0.2 exactly. Verified in this project. Money should use `Decimal` / `Numeric`.

Why the other options are wrong.

- B. 0.1 and 0.2 cannot be stored exactly as binary floats, so the sum is not exactly 0.3.
- C. Pydantic accepts any valid float, however many decimal places it has.
- D. `response_model` does not round floats. It only validates the type and filters the fields.

Q15. What does the parameter declaration below do?

HIGH
Dependency Injection

```
def create_account(account_data: AccountCreate,
                   service: AccountService = Depends()):
```

- A. Python evaluates the default once at import time, so one `AccountService` is shared by all requests
- B. FastAPI treats `service` as a required query parameter named `service`
- C. FastAPI builds an `AccountService` per request from the type hint, resolving its `Depends()`
- D. FastAPI raises an error at startup, because `Depends()` requires a callable argument

Correct answer: C. FastAPI builds an `AccountService` per request from the type hint, resolving its `Depends()`

Explanation. `Depends()` without an argument means *use the type hint*. FastAPI calls `AccountService(...)`, which needs `AccountRepository = Depends()`, which needs `get_db`.

Why the other options are wrong.

- A. FastAPI replaces the `Depends()` marker with a fresh value on every request; the default object is never used as the service.
- B. A parameter with a `Depends` default is a dependency, not a query parameter, and it does not appear in the docs.
- D. `Depends()` with no argument is valid. FastAPI falls back to the type annotation.

Q16. A developer appends this route at the end of `account_router.py` (after `@router.get("/{account_id}")`). What does `GET /accounts/summary` return?

HIGH

Routing order

```
@router.get("/summary")
def summary():
    return {"ok": True}
```

- A. `200 {"ok": true}`, because FastAPI prefers static paths over parameterised ones
- B. `422: /{account_id}` matches first and "summary" isn't an int
- C. `404`, because "summary" is not the id of an existing account
- D. `405`, because two GET routes now conflict under the same prefix

Correct answer: B. `422: /{account_id}` matches first and "summary" isn't an int

Explanation. Routes are matched in declaration order. The first match wins and its path parameter fails `int` conversion (`int_parsing`). Verified. Declare fixed paths before parameterised ones.

Why the other options are wrong.

- A. FastAPI does not reorder routes. Matching follows declaration order, so the first matching route wins.
- C. No account lookup happens. "summary" fails int conversion first, which gives 422.
- D. Two GET routes with different paths do not conflict; 405 is about the HTTP method, not route clashes.

Q17. What does `POST /accounts/` with body `{"owner_name": ""}` return?

MEDIUM

Validation

- A. `422`, because `str` fields reject empty strings by default
- B. `500`: unhandled SQLite IntegrityError
- C. `400`, because `nullable=False` rejects the empty name
- D. `200`; an account with an empty name is created

Correct answer: D. `200`; an account with an empty name is created

Explanation. An empty string is a valid `str` and is not NULL, so neither Pydantic nor the NOT NULL constraint objects. Add `Field(min_length=1)` to get a 422. Verified.

Why the other options are wrong.

- A. An empty string is still a valid `str`. Only `Field(min_length=1)` would reject it.
- B. No constraint is violated, so SQLite raises no IntegrityError.
- C. `nullable=False` only rejects NULL. An empty string is not NULL.

Q18. `bank.db` and `.env` exist only inside `banking_api/`. From the parent folder you run `uvicorn banking_api.app.main:app`. What happens? **HIGH**
Working directory

- A. `ImportError: attempted relative import with no known parent package`
- B. It starts with the "(Local Server)" name but creates a new empty `bank.db` in the parent folder
- C. It starts with the "(Local Server)" name and uses `banking_api/bank.db`, since paths are relative to `config.py`
- D. It starts, prints "Starting Personal Banking API" and creates a new empty `bank.db` in the parent folder

Correct answer: D. It starts, prints "Starting Personal Banking API" and creates a new empty `bank.db` in the parent folder

Explanation. Both `env_file=".env"` and `sqlite:///./bank.db` are resolved against the process's current working directory. From the parent, `.env` isn't found (class default name) and a fresh DB file appears there. Verified.

Why the other options are wrong.

- A. Running it as `banking_api.app.main` keeps it inside a package, so the relative imports still work.
- B. `.env` is also looked up in the working directory. From the parent folder it isn't found, so the class default name is used.
- C. Relative paths in the settings are resolved against the current working directory, not the location of `config.py`.

Q19. You add `pin_hash = Column(String)` to `Account` (on a fresh database). `GET /accounts/{account_id}` still declares `response_model=AccountOut`. What does the client receive? **MEDIUM**
Response model

- A. `500`, because the object has an attribute `AccountOut` does not declare
- B. Only `id`, `owner_name`, `balance`; `response_model` filters it
- C. All four fields, because the ORM object is returned unchanged
- D. `pin_hash` too, but only when `from_attributes=True` is removed

Correct answer: B. Only `id`, `owner_name`, `balance`; `response_model` filters it

Explanation. The return value is converted into `AccountOut`, which declares three fields; anything else is dropped. That is why output schemas protect sensitive columns.

Why the other options are wrong.

- A. Extra attributes are ignored during conversion; they do not cause an error.
- C. The return value is converted into `AccountOut`, which drops any field it doesn't declare.
- D. `from_attributes` controls how attributes are read, not which fields are output. Filtering always applies.

Q20. A buggy transfer implementation crashes between the two updates. After the request ends, what is in the database?

HIGH
Transactions

```
src.balance -= amount
self.repo.update(src)           # commits
dst.balance += amount
raise RuntimeError("crash")     # simulated failure
self.repo.update(dst)           # never reached
```

- A. `src` is unchanged, because `refresh()` inside `update()` reverts it on error
- B. Both balances are updated, because `dst` is tracked and saved when the session closes
- C. `src` is debited but `dst` is unchanged: the money disappears
- D. Both balances are unchanged, because the error rolls back the whole request

Correct answer: C. `src` is debited but `dst` is unchanged: the money disappears

Explanation. The first `commit()` is permanent. The pending change to `dst` is discarded when the session closes without a commit. Correct design: change both, then commit once.

Why the other options are wrong.

- A. `refresh()` only re-reads the committed row, which already holds the debited balance.
- B. Closing a session without committing discards pending changes; it never saves them.
- D. The first `commit()` has already made the debit permanent. Only uncommitted changes can be rolled back.

Q21. A developer changes every endpoint from `def` to `async def` but keeps the synchronous SQLAlchemy session. What is the effect?

HIGH
Sync vs async

- A. It becomes faster, because `async def` makes the database calls non-blocking
- B. It works, but blocking DB calls freeze the event loop for other requests
- C. It fails at startup, because a sync Session cannot be used inside `async def`
- D. Nothing changes, because FastAPI runs every endpoint in the thread pool anyway

Correct answer: B. It works, but blocking DB calls freeze the event loop for other requests

Explanation. FastAPI runs `def` endpoints in a thread pool but awaits `async def` endpoints directly on the event loop. Blocking I/O inside them freezes the loop. Use `async` end to end or keep `def`.

Why the other options are wrong.

- A. `async def` does not make synchronous library calls non-blocking.
- C. Sync sessions can be called from `async def`. It works, but it blocks the event loop.
- D. Only `def` endpoints are sent to the thread pool; `async def` endpoints run directly on the event loop.

Q22. The router is `APIRouter(prefix="/accounts")` and `main.py` changes to `app.include_router(account_router, prefix="/api/v1")`. What is the deposit path now?

MEDIUM
Routers

- A. `/api/v1/accounts/{account_id}/deposit`
- B. `/accounts/api/v1/{account_id}/deposit`
- C. `/accounts/{account_id}/deposit`, because the router's own prefix takes precedence
- D. `/api/v1/{account_id}/deposit`, because the include prefix replaces the router prefix

Correct answer: A. `/api/v1/accounts/{account_id}/deposit`

Explanation. Prefixes are concatenated: include prefix + router prefix + route path. Verified from the generated OpenAPI paths.

Why the other options are wrong.

- B. Prefixes are applied from the outside in, so the include prefix comes first.
- C. Both prefixes apply. Neither one overrides the other.
- D. The include prefix is added in front of the router prefix; it does not replace it.

Q23. In `AccountRepository.create()`, what is the purpose of `self.db.refresh(account)` after `commit()`?

MEDIUM
Sessions

- A. To reload the row so the object has DB values like the new `id`
- B. To validate the object against `AccountOut` before it leaves the repository
- C. `commit()` only stages changes; `refresh()` is what writes them to the file
- D. To return the connection to the pool before the response is serialised

Correct answer: A. To reload the row so the object has DB values like the new `id`

Explanation. `commit()` expires the object's attributes; `refresh()` re-reads the row immediately, including the `id` SQLite assigned.

Why the other options are wrong.

- B. The repository knows nothing about Pydantic. Validation against `AccountOut` happens in the router layer.
- C. `commit()` does write the changes. `refresh()` only reads the row back.
- D. The session, and with it the connection, is released by `get_db`'s `finally` block, not by `refresh()`.

Q24. What status code does a successful `POST /accounts/` return in this project?

MEDIUM
HTTP semantics

- A. `200`, FastAPI's default unless `status_code` is set
- B. `202`, because the commit completes after the response is sent
- C. `201`, because a `response_model` is declared on a POST route
- D. `201`, because FastAPI automatically uses Created for POST routes

Correct answer: A. `200`, FastAPI's default unless `status_code` is set

Explanation. FastAPI defaults to `200` for every method. REST convention for creation is `201`: add `status_code=201` to the decorator. Verified: the project returns `200`.

Why the other options are wrong.

- B. `commit()` finishes inside the request, before the response is sent. `202` is never sent automatically.
- C. `response_model` shapes the body, not the status code.
- D. FastAPI uses `200` for every method unless `status_code` is set on the decorator.

Q25. From inside `banking_api/` a learner runs `python app/main.py`. What happens?

MEDIUM
Python packaging

- A. It fails with `ModuleNotFoundError: No module named 'app'`
- B. It runs, creates the `app` object and exits silently without serving
- C. `ImportError: attempted relative import with no known parent package`
- D. The API starts on `127.0.0.1:8000`, exactly as with uvicorn

Correct answer: C. `ImportError: attempted relative import with no known parent package`

Explanation. Run directly, `main.py` becomes the top-level `__main__` with no package, so from `.routers...` cannot resolve. Verified. Use `uvicorn app.main:app`.

Why the other options are wrong.

- A. The error comes from the relative import (`from .routers ...`), not from a missing module named `app`.
- B. It never gets as far as creating `app`: the relative import on the second line fails first.
- D. `python app/main.py` never starts a server, and in any case the relative imports fail first.

Answer summary

Q1	B	Q2	D	Q3	B	Q4	C	Q5	D
Q6	A	Q7	D	Q8	A	Q9	B	Q10	C
Q11	D	Q12	A	Q13	C	Q14	A	Q15	C
Q16	B	Q17	D	Q18	D	Q19	B	Q20	C
Q21	B	Q22	A	Q23	A	Q24	A	Q25	C

Score bands (suggested)

Score	Reading	Suggested follow-up
22-25	Strong: understands layers, DI and failure modes	Stretch lab: transfer feature + tests
16-21	Solid on structure, gaps on edge cases	Re-run Lab 1 predictions; review status-code table
10-15	Knows the files, unclear on request flow	Pair-trace one request through all layers
0-9	Needs Python fundamentals first	Primer slide: yield, decorators, imports; then repeat labs